Software Analysis and Testing

João Saraiva

2025/2026

Departamento de Informática
Universidade do Minho, Portugal

Software Testing

Why should we test a program?!

Because ERRORS are everywhere!!

Errors Are Everywhere

Area	Error
Hearing	Spoken: He has a garage for repairing *foreign* cars.
	Heard: He has a garage for repairing *falling* cars.
Medicine	Incorrect antibiotic prescribed.
Music performance	Incorrect note played.
Numerical analysis	Incorrect algorithm for matrix inversion.
Observation	Operator fails to recognize that a relief valve is stuck open.
Software	Operator used: $\neq$, correct operator: $>$.
	Identifier used: new_line, correct identifier: next_line.
	Expression used: $a \wedge (b \vee c)$, correct expression: $(a \wedge b) \vee c$.
	Data conversion from 64-bit floating point to 16-bit integer not protected (resulting in a software exception).
Speech	Spoken: *waple malnut*, intent: *maple walnut*.
	Spoken: *We need a new refrigerator*, intent: *We need a new washing machine*.
Sports	Incorrect call by the referee in a tennis match.
Writing	Written: What kind of *pans* did you use?
	Intent: What kind of *pants* did you use?

Software Errors in Practice...

It took the European Space Agency 10 years and $7 billion to produce Ariane 5, a giant rocket capable of hurling a pair of three-ton satellites into orbit with each launch and intended to give Europe overwhelming supremacy in the commercial space business..

All it took to explode that rocket less than a minute into its maiden voyage last June, scattering fiery rubble across the mangrove swamps of French Guiana, was a small computer program trying to stuff a 64-bit number into a 16-bit space.

http://www.around.com/ariane.html

Software Errors in Space...

CNN.com

sci-tech> space> story page

MAIN PAGE
WORLD
U.S.
LOCAL
POLITICS
WEATHER
BUSINESS
SPORTS
TECHNOLOGY
SPACE
HEALTH
ENTERTAINMENT
BOOKS
TRAVEL
FOOD
ARTS & STYLE
NATURE
IN-DEPTH
ANALYSIS
myCNN

Headline News brief
news quiz
daily almanac

MULTIMEDIA:
video
video archive
audio
multimedia showcase
more services

banner

Metric mishap caused loss NASA orbiter

September 30, 1999
Web posted at: 4:21 p.m. EDT (2021 GMT)

Orbiter

By Robin Lloyd
CNN Interactive Senior Writer

(CNN) -- NASA lost a $125 million Mars orbiter because a Lockheed Martin engineering team used English units of measurement while the agency's team used the more conventional metric system for a key spacecraft operation, according to a review finding released Thursday.

Martin engineering team used English units of measurement while the agency's team used the more conventional metric system for a key spacecraft operation, according to a review finding released Thursday.

Software Errors in the Olympic Games…

Around 200 people who thought their only experience of the London 2012 Olympic Games would be minor heats of synchronised swimming have received an unexpected upgrade to the men's 100m final following an embarrassing ticketing mistake.

…

Locog said the error occurred in the summer, between the first and second round of ticket sales, when a member of staff made a single keystroke mistake and entered ***'20,000' into a spreadsheet rather than the correct figure of 10,000***

The Telegraph, 04 January 2012

The Telegraph Monday 15 July 2013

LONDON2012

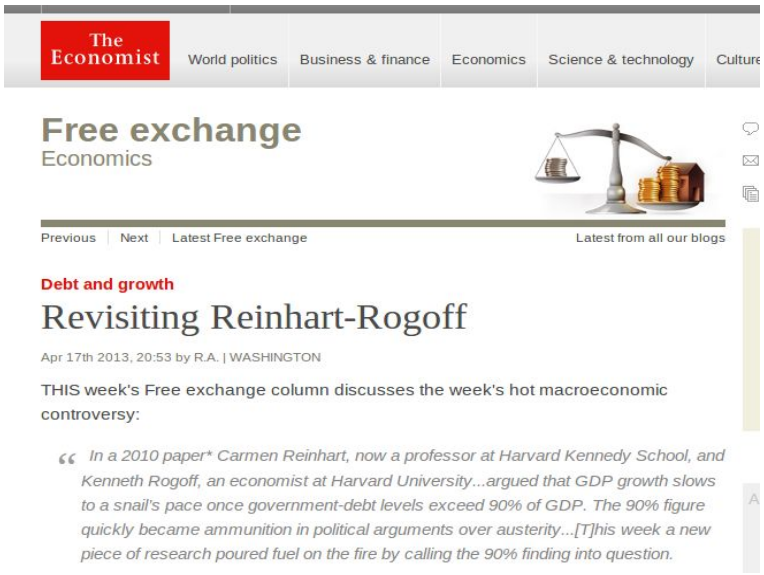
HOME | PARALYMPICS | SCHEDULE | MEDALS | SPORT GUIDES | PICTURE GALLERIES | VENUES

HOT TOPICS: Paralympic magic moments | Paralympic medal map | Classifications explained | 50 best imag

London 2012 Olympics: lucky few to get 100m final tickets after synchronised swimming was overbooked by 10,000

Around 200 people who thought their only experience of the London 2012 Olympic Games would be minor heats of synchronised swimming have received an unexpected upgrade to the men's 100m final following an embarrassing ticketing mistake.

Software Errors in our Lives...


The Economist | World politics | Business & finance | Economics | Science & technology | Culture

Free exchange
Economics

Previous | Next | Latest Free exchange | Latest from all our blogs

Debt and growth

Revisiting Reinhart-Rogoff

Apr 17th 2013, 20:53 by R.A. | WASHINGTON

THIS week's Free exchange column discusses the week's hot macroeconomic controversy:

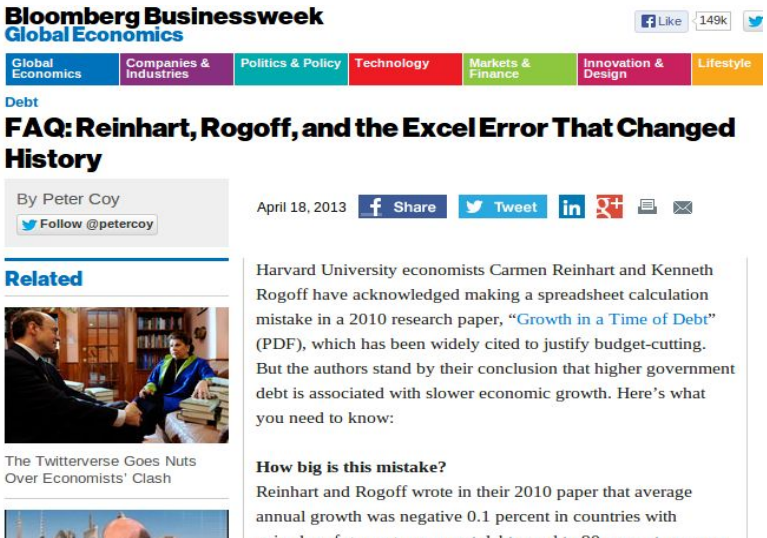
> *In a 2010 paper* Carmen Reinhart, now a professor at Harvard Kennedy School, and Kenneth Rogoff, an economist at Harvard University...argued that GDP growth slows to a snail's pace once government-debt levels exceed 90% of GDP. The 90% figure quickly became ammunition in political arguments over austerity...[T]his week a new piece of research poured fuel on the fire by calling the 90% finding into question.*

In a 2010 paper Carmen Reinhart, now a professor at Harvard Kennedy School, and Kenneth Rogoff, an economist at Harvard University...argued that GDP growth slows to a snail's pace once government-debt levels exceed 90\% of GDP. The 90% figure quickly became ammunition in political arguments over austerity...This week a new piece of research poured fuel on the fire by calling the 90% finding into question..*

The Economist, 17 April 2

Harvard University economists Carmen Reinhart and Kenneth Rogoff have acknowledged making a spreadsheet calculation mistake in a 2010 research paper, "Growth in a Time of Debt", which has been widely cited to justify budget-cutting.

Business Week, 18 April 2013


Bloomberg Businessweek
Global Economics

Like 149k

Global Economics | Companies & Industries | Politics & Policy | Technology | Markets & Finance | Innovation & Design | Lifestyle

Debt

FAQ: Reinhart, Rogoff, and the Excel Error That Changed History

By Peter Coy
Follow @petercoy

April 18, 2013 Share Tweet

Related

The Twitterverse Goes Nuts Over Economists' Clash

Harvard University economists Carmen Reinhart and Kenneth Rogoff have acknowledged making a spreadsheet calculation mistake in a 2010 research paper, "Growth in a Time of Debt" (PDF), which has been widely cited to justify budget-cutting. But the authors stand by their conclusion that higher government debt is associated with slower economic growth. Here's what you need to know:

How big is this mistake?

Reinhart and Rogoff wrote in their 2010 paper that average annual growth was negative 0.1 percent in countries with

Software Errors in our Lives...

BBC | Sign in | Home | News | Sport | Reel | Worklife | Travel | Fut

NEWS

Home | US Election | Coronavirus | Video | World | UK | Business | Tech | Science | Stories | Entertainment & Arts

Tech

Excel: Why using Microsoft's tool caused Covid-19 results to be lost

By Leo Kelion
Technology desk editor

5 October

Coronavirus pandemic

The badly thought-out use of Microsoft's Excel software was the reason nearly 16,000 coronavirus cases went unreported in England.

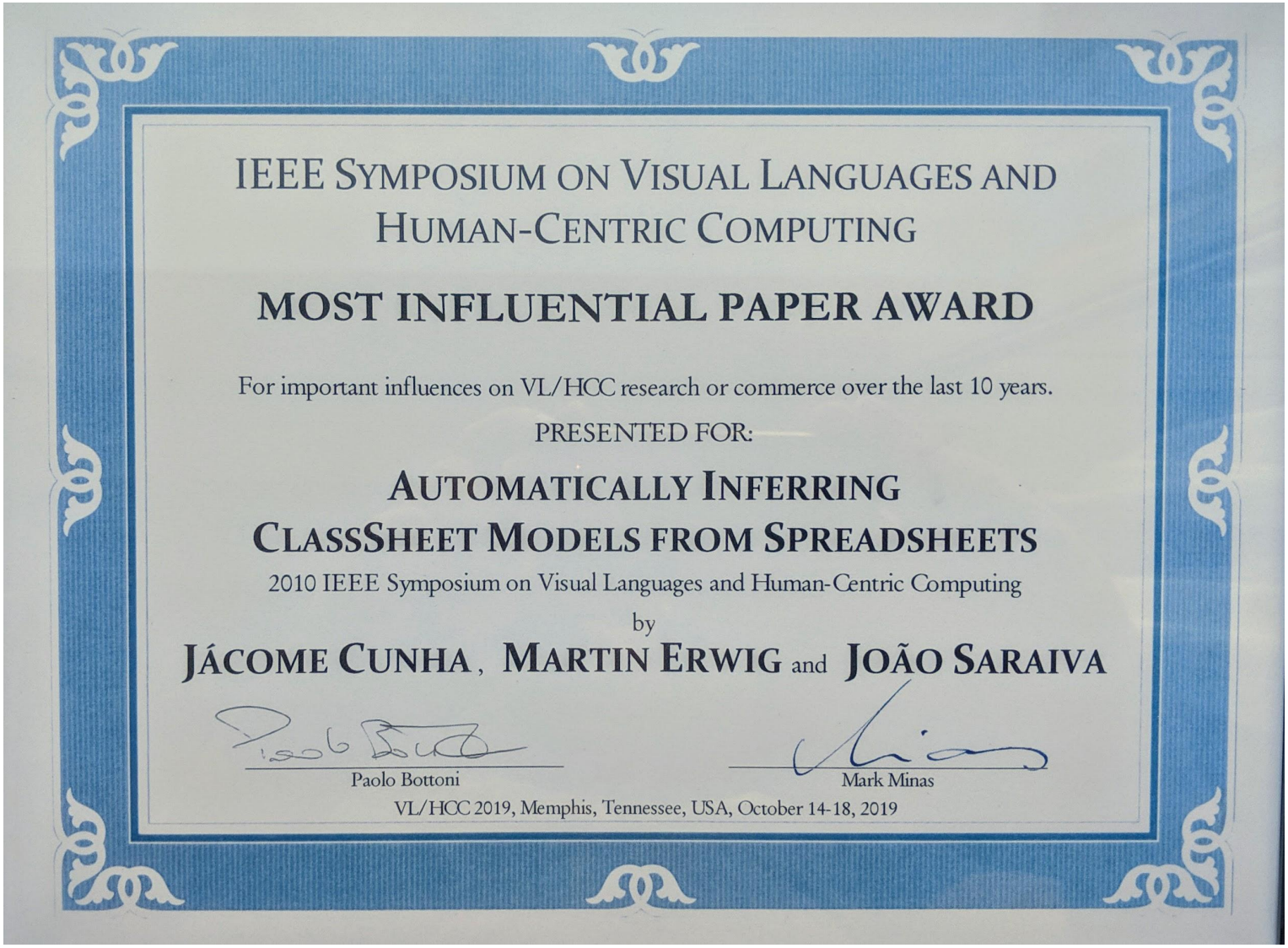
And it appears that Public Health England (PHE) was to blame, rather than a third-party contractor.

The issue was caused by the way the agency brought together logs produced by commercial firms paid to analyse swab tests of the public, to discover who has the virus.

They filed their results in the form of text-based lists - known as CSV files - without issue.

PHE had set up an automatic process to pull this data together into Excel templates so that it could then be uploaded to a central system and made available to the NHS Test and Trace team, as well as other government computer dashboards.

Avoiding Errors in Spreadsheets :-)



IEEE SYMPOSIUM ON VISUAL LANGUAGES AND HUMAN-CENTRIC COMPUTING

MOST INFLUENTIAL PAPER AWARD

For important influences on VL/HCC research or commerce over the last 10 years.

PRESENTED FOR:

AUTOMATICALLY INFERRING CLASSSHEET MODELS FROM SPREADSHEETS

2010 IEEE Symposium on Visual Languages and Human-Centric Computing

by

JÁCOME CUNHA, MARTIN ERWIG and JOÃO SARAIVA

Paolo Bottoni

Mark Minas

VL/HCC 2019, Memphis, Tennessee, USA, October 14-18, 2019

Testing Questions

- Should I test my own code, or should somebody else?

- Which code of my project should I test the most/least?

- Can I test all possible inputs to see whether something works?

- How do I know if I've tested the program adequately?

- What constitutes a good or bad test case method?

- Is it good or bad if a test case fails?

- What if a test case itself has a bug in it?

Software Testing: Why?

- **Risk Mitigation** – Testing helps identify defects and failures early in the development process when they are less expensive to fix. This reduces project risks related to quality, security, performance, etc.
- **Confidence** – Executing a well–planned software test strategy provides confidence that the software works as intended before its release.
- **Compliance** – Testing can ensure that software adheres to standards, regulations, and compliance requirements. This is especially critical for safety–critical systems.
- **User Satisfaction** – Rigorous testing from a user perspective can verify usability, functionality, and compatibility. This increases customer/user satisfaction and reduces negative impacts to an organization's reputation or finances from poor quality products.
- **Optimization** – Testing provides vital feedback that can be used to continuously improve software quality, user experience, security, performance and other product attributes.
- **Cost Savings** – Investing in testing activities reduces downstream costs related to defects found post–release. It is much cheaper to find and fix bugs earlier in the development cycle.

Software Testing: Types

- **Black box vs. white box** — Black box testing focuses on inputs/outputs without internal logic visibility while white box testing leverages internal code structures.

- **Random vs. systematic** – Systematic techniques rigorously create tests while random approaches generate arbitrary data.

- **Static vs. dynamic** – Static techniques like reviews analyze code without execution while dynamic techniques involve code execution.

- **Model–based** – Models of the software guide test generation and oracles for verification.

Software Testing

Black and White-Box Testing

Black-Box Testing

black-box test: Written without knowledge of how the program (or software component) under test is implemented.

example: Mooshak. Java project of ATS 2016!

Black-Box Testing

black-box test: Written without knowledge of how the program (or software component) under test is implemented.

example: Mooshak.
Java project of ATS 2016!

testing mobile apps!

Black-box Testing

- **Black-box testing** is based on requirements and functionality, not code

- Tester may have actually seen the code before ("gray box") but doesn't look at it while constructing the tests

- Often done from the end user or software developer's perspective

Emphasis on parameters, inputs/outputs (and their validity)

Types of Black-box Testing

- Requirements based
  → test requirements
  example: should the ***nub*** function preserve order?
- Positive/negative results
  → checks both good/bad results
- Boundary value analysis
- Equivalence partitioning
  → group related inputs/outputs
- Compatibility testing
- User documentation testing
- Domain testing

Boundary Testing

boundary value analysis: Testing conditions on bounds between classes of inputs.

Why is it useful to test near boundaries?

- likely source of programming errors (< vs. <=, etc.)
- language has many ways to implement boundary checking
- requirement specs may be fuzzy about behavior on boundaries

Equivalence Testing

Equivalence partitioning: A black-box test technique to reduce the number of required test cases.

Steps in equivalence testing:

1. Identify classes of inputs with same behavior.
2. Test on at least one member of each equivalence class.
3. Assume behavior will be same for all members of class

Equivalence Testing

criteria for selecting equivalence classes:

coverage : every input is in one class
disjointedness : no input in more than one class
representation : if error with 1 member of class,
will occur with all

White-box Testing

White-box testing: Written with knowledge of the implementation of the source code under test.

- focuses on internal states of objects and code
- focuses on trying to cover all code paths/code blocks/ statements
- requires internal knowledge of the component to craft input

example: knowing that a program uses an array of size 256 as its internal data structure test with 255 or 257 values.

Layers of Software Testing

Phase	Technique
Coding	Unit testing
Integration	Integration testing
System integration	System testing
Maintenance	Regression testing
Post system, pre-release	Beta-testing

Layers of Software Testing

Unit Test: A software testing technique in which small code units are tested independently to determine if they execute correctly.

Integration Test: A software testing technique where code units are combined and tested as a whole. Typically occurs after unit testing, to determine if the tested code units also execute correctly when integrated into a single software system.

(NASA Mars orbiter device…)

Layers of Software Testing

System Test: A testing layer where the complete, integrated software system is tested to check if it behaves correctly, that is, if it matches the specified requirements.

Regression Test: A software testing technique used to check if already tested software keeps on working properly after being changed. The changes can result from the natural evolution of the software after a change in the requirements, after refactoring, etc.

Layers of Software Testing

Beta Testing: Considered the last layer of software testing, consists in testing the software in a real environment. It usually entails distributing the software to *"beta test sites"* and users (*"beta testers"*) who are no involved in its development, to expose it to "real world" behaviour.

What are the key issues in Sw Testing?

Some key issues and challenges in effective software testing include:

- **Adequate test coverage** – Testing everything is not feasible so strategies are needed to maximize coverage and detection of critical defects.
- **Test effort and scheduling** – Testing consumes significant time and resources so it must be properly scheduled.
- **Selecting effective techniques** – There are many testing techniques and choosing appropriate ones requires expertise (Unit Testing, Mutation Testing, Property-based Testing, etc)
- **Automation** – Executing manual testing is tedious so automation solutions are needed but require investment.
- **Non–functional aspects** – Testing quality attributes like security and performance (time, memory and energy consumptions) can be challenging.

What are the key issues in Sw Testing?

- **Test environment** – Testing should mimic real–world environments, but this can be costly and or complex to emulate.
- **Defect diagnostics** – Isolating the root causes of failures can be difficult and time–consuming.
- **Testability** – Code and overall architecture should be optimized for testing as much as possible.
- **Interoperability** – Testing interfaces between systems and integration points can quickly become complex.
- **Scalability & performance** – Testing distributed and high–performance systems often poses challenges, especially given that some types of fault may not produce failures until the system is running at scale.

Software Testing

Source Code Coverage

White Box Testing: An Example

How do you test this program?

T = { t_1 : (a = 1, b = 1,c=2)
, t_2 : (a = 3, b = 4, c=5)
}

t_3 : (a = 2, b = 4, c=0) ?

```
#include <stdio.h>
int main()
{ int a,b,c;
  printf(" Introduza a,b,c\n");
  scanf("%d%d%d",&a,&b,&c);

  if ( (a == 3) || (aux(b,c) > 2) )
    printf("Ramo 1");
  else printf("Ramo 2");

  return 0;
}

int aux(int x, int y)
{ return (x/y); }
```

Source Code Coverage Testing

Source code coverage testing: It is a white-box testing technique that examines what fragments of the source code under test are executed by existing test cases.

- Statement coverage
  - Tries to reach every program statement
- Path coverage
  - Follow every distinct brach through code
- Condition coverage
  - Every condition that leads to a branch
- Function/method coverage
  - Treat every behavior/end goal separately

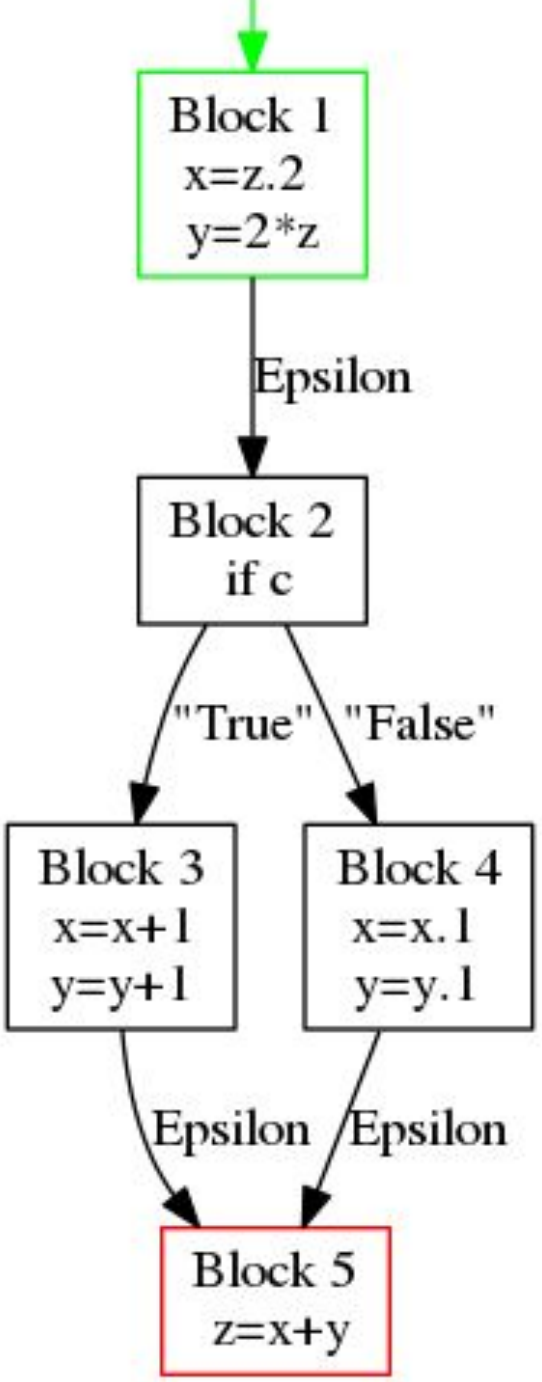
Control Flow Graphs (CFG)

A control flow graph G is a graph defined as a finite set N of nodes and a finite set E of edges, where an edge (i, j) in E connects two nodes n_i and n_j in N.

We often write G= (N, E) to denote a flow graph G with nodes given by N and edges E.

Control Flow Graph - Example

N = { 1, 2, 3 , 4 , 5 }
E = { (1,2) , (2,3) , (2,4) ,
(3,5), (4,5) }



Control Flow Graph - Paths

Consider a CFG G= (N, E).

A sequence of k edges, with k>0, (e_1 , e_2 , ... e_k), is a path of length k through the control flow graph if and only if e_i = (n_p , n_q) and e_{i+1} = (n_r , n_s), with 0< i < k, and n_p, n_q, n_r, and n_s in N, and n_q = n_r.

There are two specially designated blocks: the *entry block*, through which control enters into the flow graph, and the *exit block*, through which all control flow leaves

Unreachable node: there is no path from the entry block to that node.

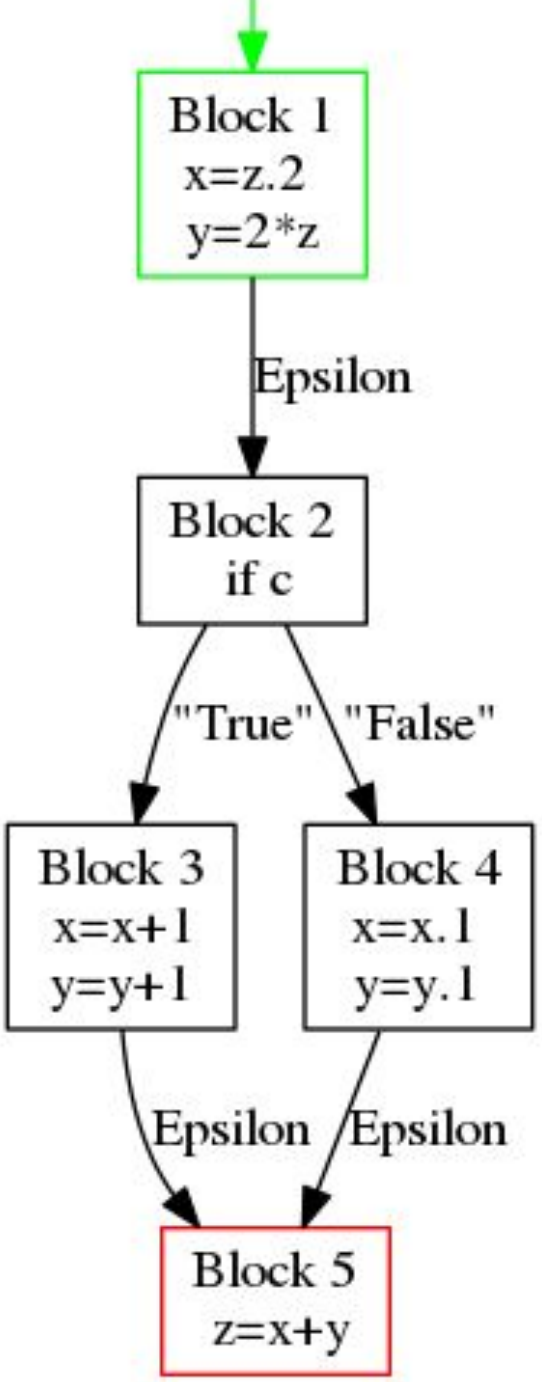
Control Flow Graph - Paths

Two complete paths:

P1 = ((1,2) , (2,3) , (3,5))
P2 = ((1,2) , (2,4) , (4,5))

Both paths have length 3.

TPC: write a function to compute all complete paths for a given CFG.



Control Flow Graph – Number of Paths

A program can contain many distinct paths! The number of paths is often used as a metric of the complexity of a program: ***Cyclomatic Complexity***.

A program with no condition contains exactly one path. Each additional condition in the program can increases the number of distinct paths by at least one.

Depending on their location, conditions can have a multiplicative effect on the number of paths.

– nested if-then-else

– while/for loop

Control Flow Graph – Cyclomatic Complexity

Definition

There are multiple ways to define cyclomatic complexity of a section of source code. One common way is the number of linearly independent paths within it. A set S of paths is linearly independent if the edge set of any path P in S is not the union of edge sets of the paths in some subset of S/P. If the source code contained no control flow statements (conditionals or decision points) the complexity would be 1, since there would be only a single path through the code. If the code had one single-condition IF statement, there would be two paths through the code: one where the IF statement is TRUE and another one where it is FALSE. Here, the complexity would be 2. Two nested single-condition IFs, or one IF with two conditions, would produce a complexity of 3.

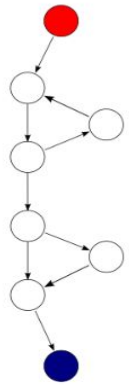
Control Flow Graph – Cyclomatic Complexity

Another way to define the cyclomatic complexity of a program is to look at its control-flow graph, a directed graph containing the basic blocks of the program, with an edge between two basic blocks if control may pass from the first to the second. The complexity M is then defined as[2]

$$M = E - N + 2P,$$

where

- E = the number of edges of the graph.
- N = the number of nodes of the graph.
- P = the number of connected components.



A control-flow graph of a simple program. The program begins executing at the red node, then enters a loop (group of three nodes immediately below the red node). Exiting the loop, there is a conditional statement (group below the loop) and the program exits at the blue node. This graph has nine edges, eight nodes and one connected component, so the program's cyclomatic complexity is $9 - 8 + 2{\times}1 = 3$.

Control Flow Graph - Paths

Feasible paths: A path p through a flow graph for program P is considered feasible if there exists at least one test case c which when running P with input c causes path p to be traversed.

Infeasible paths: A path p through a flow graph for program P is considered infeasible if there is no test case c which when running P with input c causes path p to be traversed.

Infeasible Paths

Are there infeasible paths in programs?!

- Dead Code (usually caused by program bugs):

```
(1)    Program sl
(2)    {   ...
(3)        if (a > 10) then
(4)           if (a < 3) then  // bug: 30?!
(5)
(6)            ...
(7)
(8)    }
```

Infeasible Paths

- Defensive code

```
(1)    …
(2)    switch ( ){   ...
(3)        if (a > 10) then
(4)            if (a < 3) then   // bug: 30?
(5)
(6)          ...
(7)
(8)    }
```

Source Code: Statement Coverage

The **statement coverage** of T with respect to program **P** is computed as

$$S_c / (S_e - S_i)$$

where

- S_c is the number of statements covered,
- S_i is the number of unreachable statements, and
- S_e is the total number of executable statements in program **P** (the size of the coverage domain).

T is adequate with respect to the statement coverage criterion if the statement coverage of T with respect to program **P** is 1.

Statement Coverage: Example

Statemente Coverage Domain: S_e = { 3,4,5, 6,7,8,9,11,12}

T_1 = { t_1 : (x = −1, y = −1)
, t_2 : (x = 1, y = 1)
}

statements covered
t_1 : 3,4,5,6,7,8,12
t_2 : 3,4,5,6,11,12
S_c = 8

T_1 is adequate for **ex1**
since 8 / (9-1) = 1

(stat 9 is dead)

```
(1)  Program ex1
(2)  { int a,b,c;
(3)    scan(a);
(4)    scan(b);
(5)    c = 0;
(6)    if (a<0 && b<0)then
(7)     { c = x * 2;
(8)       if (a>=0) then
(9)         c = c + 1; }
(10)   else
(11)      c = x *3;
(12)   print (c);
(14)  }
```

Source Code: Block Coverage

The **block coverage** of T with respect to program **P** is computed as

$$B_c / (B_e - B_i)$$

where

- B_c is the number of blocks covered,
- B_i is the number of unreachable blocks, and
- B_e is the total number of blocks in program **P** (the size of the block coverage domain).

T is adequate with respect to the block coverage criterion if the block coverage of T with respect to program **P** is 1.

Source Code: Block Coverage

The **block coverage** of T with respect to program **P** is computed as

$$B_c / (B_e - B_i)$$

where

- B_c is the number of blocks covered,
- B_i is the number of unreachable blocks, and
- B_e is the total number of blocks in program **P** (the size of the block coverage domain).

T is adequate with respect to the block coverage criterion if the block coverage of T with respect to program **P** is 1.

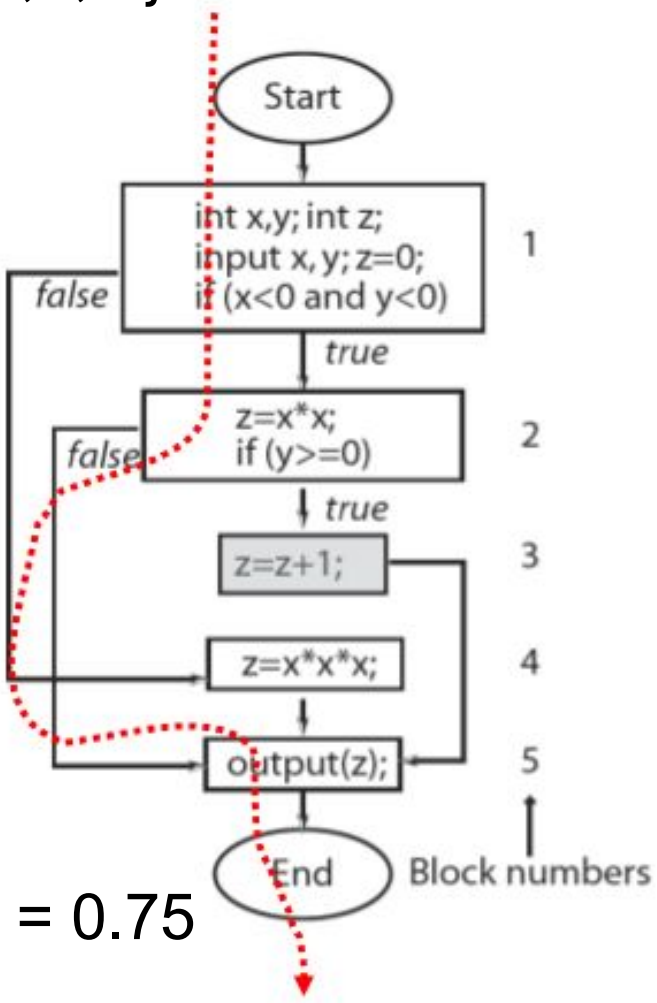
Block Coverage: Example

Block Coverage Domain: B_e = { 1,2,3,4,5 }

T_2 = { t_1 : (x = −1, y = −1)
, t_2 : (x = -3, y = -1)
, t_3 : (x = -1, y = -1)
}

Blocks covered
t_1 : 1,2,5
t_2 : 1,2,5
t_3 : 1,2,5

B_c = 3



T_2 is not adequate for **ex1** since 3 / (5-1) = 0.75
(block 3 is dead)

Code Coverage: Path Coverage

The **path coverage** of T with respect to program **P** is computed as

$$P_c / (P_e - P_i)$$

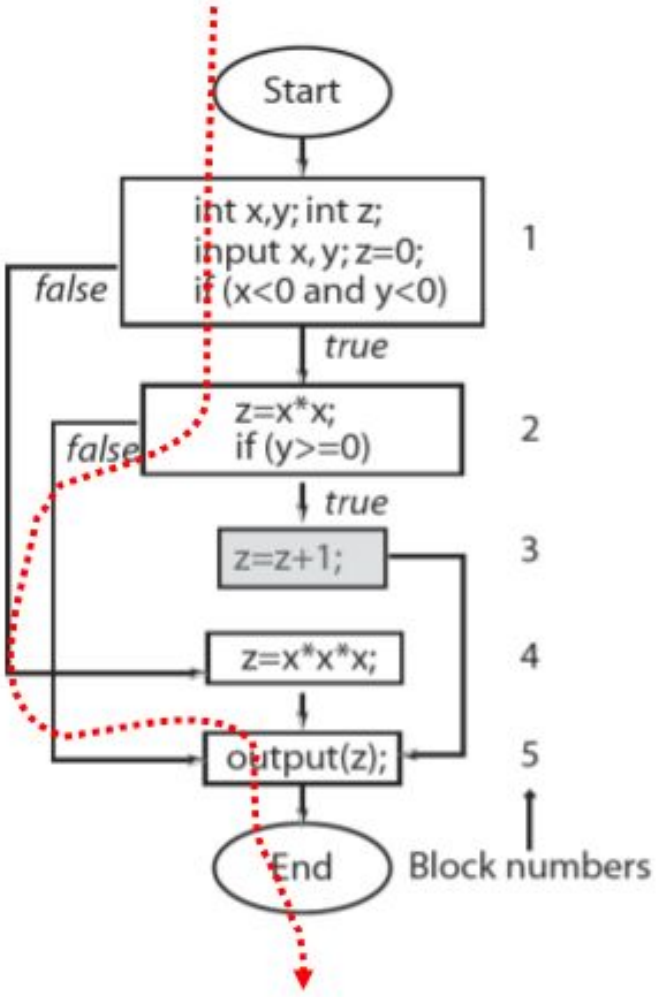
where
- P_c is the number of paths covered,
- P_i is the number of unfeasible path, and
- P_e is the total number of feasible paths in program **P**

T is adequate with respect to the path coverage criterion if the path coverage of T with respect to program **P** is 1.

Path Coverage: Example

Exercise 1: Define the sets of feasible and unfeasible paths of program ex1.

Exercise 2: Define a set of test cases that cover **ex1** adequately with respect to the path criterion.



Source Code: Block Coverage

The **block coverage** of T with respect to program **P** is computed as

$$B_c / (B_e - B_i)$$

where

- B_c is the number of blocks covered,
- B_i is the number of unreachable blocks, and
- B_e is the total number of blocks in program **P** (the size of the block coverage domain).

T is adequate with respect to the block coverage criterion if the block coverage of T with respect to program **P** is 1.

Code Coverage: Condition Coverage

A decision can be composed of a **simple condition** such as **x < 0**, or of a more complex condition, such as **((x < 0 && y < 0) || (p ≥ q))**.

&& and || are the logical operators that connect two or more simple conditions to form a **compound condition**.

A **simple condition** is considered covered if it evaluates to true **and** false in one or more executions.

A **compound condition** is considered covered if each simple condition it is comprised of is also covered.

Code Coverage: Condition Coverage

Decision coverage is concerned with the coverage of decisions regardless of whether or not a decision corresponds to a simple or a compound condition.

```
(1) if (a < 0 && b < 0) then
(2)     c = foo(a, b);
```

How many decision?

There is only one decision that leads control to line 2 if the compound condition inside the **if** evaluates to true.

Code Coverage: Condition Coverage

However, a compound condition might evaluate to true or false in one of several ways.

How many decision?

```
(1) if (x < 0) then
(2)     if (y < 0) then
(3)         z=foo(x, y);
```

Two decisions: one in each if statement

Source Code: Condition Coverage

The **condition coverage** of T with respect to program **P** is computed as

$$C_c / (C_e - C_i)$$

where

- C_c is the number of simple conditions covered,
- C_i is the number of infeasible simple conditions,
- C_e is the total number of simple conditions in program **P**.

T is adequate with respect to the condition coverage criterion if the conditions coverage of T with respect to program **P** is 1.

Condition Coverage: Switch Statements

Decision implied by the switch statement is considered covered if during one or more executions of the program under test the flow of control has been diverted to all possible destinations.

Condition Coverage: Example

Exercise: write a switch statement and an equivalent if-the-else version on that fragment and compute the (path) coverage in a tool/IDE that offers it.

Software Testing

Unit Testing

Unit Testing

All modern programming languages offer one (or more) framework for Unit Testing:

- Java : **jUnit**
- Haskell: **HUnit**
- C# : **xUnits**

etc

Unit Testing: Ingredients

Test case: The simplest case of a unit test.

Assertion: Mechanism that assesses whether a test case produces the expected result.

Test Fixture: a fixed state used as a baseline for running tests. The purpose of a test fixture is to ensure that there is a well known and fixed environment in which tests are run so that results are repeatable.
Frequently fixtures are created by handling **setUp** and **tearDown** events of the unit testing framework.

Unit Testing: Ingredients

Test Suite: A collection of test cases that are intended to be used to test a program to show that it has some specified set of behaviour. Usually the test case share a common test fixture.

Test Runner: A framework to execute the tests, tipically involving:

1. *Fixture setup*
2. *Execution of test suite*
3. *Fixture Teardown*

Software Testing

Unit Testing Frameworks And Source Code Coverage Tools

Source Code Coverage Tools

***There are several nice tools exist for checking code coverage**. In the Lab Class we wil use*

***Cobertura, JaCoCo** (Java + IntelliJ)*
***HPC** (Haskell)*

Software Testing

Mutation Testing

Mutation Testing

- **How can we assess the quality of our test suite?**

  via coverage?

Is coverage a good quality indicator?

T = { t_1 : (a = 1, b = 1,c=2)
, t_2 : (a = 3, b = 4, c=5)
}

But the program fails… with test case:

t_3 : (a = 2, b = 4, c=0)

```
#include <stdio.h>
int main()
{ int a,b,c;
  printf(" Introduza a,b,c\n");
  scanf("%d%d%d",&a,&b,&c);

  if ( (a == 3) || (aux(b,c) > 2) )
    printf("Ramo 1");
  else printf("Ramo 2");

  return 0;
}

int aux(int x, int y)
{ return (x/y); }
```

Mutation Testing

- **How can we assess the quality of our test suite?**

  mimicking the car industry by injecting a fault on the (software) system under test.

Mutation Analysis and Testing

- **Mutant:**

  A valid (in terms of compilation) program that behaves differently than the original one.

  Usually a **small** and **local change** to a program

  A test case t **kills a mutant** m if t produces a different result on m than the original program.

Mutation Analysis and Testing

- **Basic Idea: Fault-based Testing**
  - Take a program and its test cases.
  - Systematically generate a number of similar programs (*mutants*), each different from the original in one small way, i.e., each possessing a fault.
  - The mutants are then executed with the original test data.
  - If test cases detect all different mutants, then the mutants are said to be *dead (killed)*, and the test set is adequate.

Mutation Analysis and Testing

What are possible mutants? Which are the test cases that kill the mutants?

```
int f (int a , int b)
{  if ( a > 10)
      return a + b;
   else
      return b;
}
```

Mutation Analysis and Testing

What are possible mutants?

```
// original
int f (int a , int b)
{  if ( a > 10)
      return a + b;
   else
      return b;
}
```

```
// mutant
int f (int a , int b)
{  if ( a < 10)
      return a + b;
   else
      return b;
}
```

Give a test case that kills this mutant? And a test case that does not kill this mutant?

Once we have a test case that kills a mutant, the mutant itself is no longer useful.

Mutation Analysis and Testing

Game Time!

```
// original
int f (int a , int b)
{  if ( a > 10)
       return a + b;
   else
       return b;
}
```

One half of the class:

Define a new mutant
(do not show it)

The other half:

Define test cases until the mutant is killed

Then, change roles!

The Winner: who defines lowest number of tests

Mutation Analysis and Testing

Exercise: Given the Java Class ??? and its X mutants

write the smallest test suite that kills all mutants.

(test class with setup method only)

Mutation Analysis and Testing

What are possible mutants?

Some are not generally useful:

- Not compilable (still born, *kill by compiler*)
- Killed by most test cases (trivial)
- Indistinguishable from original code (equivalent)

  equivalent mutants are a practical problem

  → It is in general **an undecidable problem**

- Indistinguishable from other mutants (redundant)

Mutation Analysis and Testing

Equivalent Mutant:

```
int f (int a , int b)
{  if ( a > 10)
       return a + b;
   else
       return b * 1;
}
```

Mutation Testing

Mutation testing is conceptually quite simple.

Faults (or mutations) are automatically introduced into the program's source code, then your tests are run. If your tests fail then the mutation is killed, if your tests pass then the mutation lived.

The quality of your tests can be assessed by the percentage of mutations killed.

The changes in mutant program are kept *extremely small*, so it does not affect the overall objective of the program.

Mutation Testing

Why Mutation Testing?

Traditional test coverage (i.e line, statement, block, path etc) measures only which code is executed by your tests.

It does not check that your tests are actually able to detect faults in the executed code. It is therefore only able to identify code that is definitely not tested.

The goal of mutation testing is to assess the quality of the test cases which should be robust enough to fail mutant code. This method is also called as Fault Based Testing as it involves injecting faults in the program.

Mutation Analysis and Testing

- A mutant remains *live* either
  - Because it is equivalent to the original program. (semantically identical although syntactically different)
  - The test set is inadequate to kill the mutant.

  In the latter case, the test data need to be augmented (by adding one or more new test cases) to kill the live mutant.
  - For the autumated generation of mutants, we use ***mutation operators***, that is predefined program modification rules.

Mutation-based Testing: The Process

- Introduce faults (mutants) into the code.
- Run the test suite against the mutants.
- Analyze which mutants are killed.
- Improve the test suite based on surviving mutants.

Mutation Analysis and Testing

Example of Mutation Operators I

- Constant replacement
- Scalar variable replacement
- Scalar variable for constant replacement
- Constant for scalar variable replacement
- Array reference for constant replacement
- Array reference for scalar variable replacement
- Constant for array reference replacement
- Scalar variable for array reference replacement
- Array reference for array reference replacement
- Source constant replacement
- Data statement alteration
- Comparable array name replacement
- Arithmetic operator replacement
- Relational operator replacement
- Logical connector replacement
- Absolute value insertion
- Unary operator insertion
- Statement deletion
- Return statement replacement

Mutation Analysis and Testing

Example of Mutation Operators II

Specific to object-oriented programming languages:

- Replacing a type with a compatible subtype (inheritance)
- Changing the access modifier of an attribute, a method
- Changing the instance creation expression (inheritance)
- Changing the order of parameters in the definition of a method
- Changing the order of parameters in a call
- Removing an overloading method
- Reducing the number of parameters
- Removing an overriding method
- Removing a hiding Field
- Adding a hiding field

Mutation Analysis and Testing

- Ideally, we would like the mutation operators to be representative of all realistic types of faults that may occur in practice.
- Mutation operators differ between languages, although there is significant overlap.
- In general, the number of mutation operators is large as they are supposed to capture all possible syntactic variations in a program.

Mutation coverage/mutation score: Complete coverage equals to killing all non-equivalent mutants.

Mutation Analysis and Testing

- **The goal it to**
  - Mutation Analysis: Measure fault finding ability.
  - Mutation Testing: Create a test suite that kills a representative set of mutants.

Mutation Analysis and Testing

```
int min(int a,int b)
{ int minVal;
  minVal = a;
  if ( b < a)
     minVal = b;
  return minVal;
}
```

Mutant 1:

```
int min(int a,int b)
{ int minVal;
  minVal = b;
  if ( b < a)
     minVal = b;
  return minVal;
}
```

Mutant 2:

```
int min(int a,int b)
{ int minVal;
  minVal = a;
  if ( b < a)
     minVal = a;
  return minVal;
}
```

Mutant 3:

```
int min(int a,int b)
{ int minVal;
  minVal = a;
  if ( b < minVal)
     minVal = b;
  return minVal;
}
```

Mutation Analysis and Testing

Test Suite:

min(1,2) → 1
min(2,1) → 1

Which mutants are killed? Which is the mutation score?

Mutation Testing: https://pitest.org

Pitest Quickstart FAQ Downloads Links About

Real world mutation testing

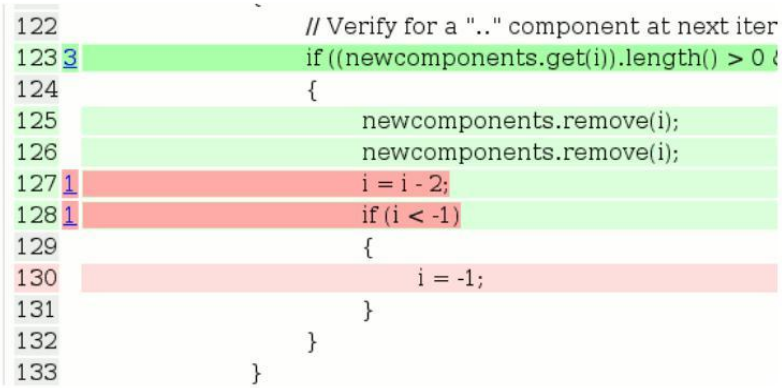
PIT is a state of the art mutation testing system, providing gold standard test coverage for Java and the jvm. It's fast, scalable and integrates with modern test and build tooling.

Mutation Testing

PIT is different. It's

- fast - can analyse in minutes what would take earlier systems days
- easy to use - works with ant, maven, gradle and others
- actively developed
- actively supported

The reports produced by PIT are in an easy to read format combining line coverage and mutation coverage information.



Example snippet taken from coverage report of Wicket Core

Light green shows line coverage, dark green shows mutation coverage.

Light pink show lack of line coverage, dark pink shows lack of mutation coverage.

Software Testing Gamification

Gamification

”The use of elements of game-playing in another activity, usually in order to make that activity more interesting.” - Oxford Dictionary

Popular strategies include:

- Rewarding users who complete tasks (virtual currencies, points, badges, levels)
- Public scoreboards
- Growing levels of difficulty

Mutation Testing